Sesiones Enterprise JavaBeans

App Java Enterprise

Incluye:

- Proyecto de una Aplicación Web
- Diseño de la interfaz de usuario
- Modelo de Datos
- Conexión a MySQL
- Generación de Entidades
- **Generación de Sesiones Beans**
- Diseño con patrones, Model-View-Controller (MVC) y Session Facade

Introducción

EJB ?

- Componente (clase java) del servidor.
- Encapsulan la lógica del negocio.
- Posee una pila integrada para:
  - Acceso remoto,
  - Mensajería,
  - Servicio web (SOAP, REST), etc.

Introducción

EJB ?

- **Modelo** muy simple (hoy en día) para desarrollar aplicaciones distribuidas, transaccionales, seguras y portátiles.
- Un simple POJO (Plain Old Java Object) mas anotaciones, podrá desplegarse en un **contenedor EJB**.

Modelo del componente EJB

EJB define tres tipos:

- *Session beans: Operan en contextos transaccionales y* distribuidos.
- *Message-driven beans* (MDBs): *responde a eventos externos.*

Complementariamente, JPA define:

- *Entidades: clases que representan tablas de una bd.*

Contenedor EJB

- Es el entorno donde operan los componentes EJB.
- Ofrece servicios:
  - de transacción y de seguridad,
  - pool de recursos y almacenamiento,
  - soporte de concurrencia, etc,

Sesiones Beans

- El nombre *sesión* implica que una instancia del bean está disponible durante una "unidad de trabajo".
- Una sesión Bean Enterprise es invocada por un cliente con el fin de realizar una operación de negocio específica.

Especificación EJB 3.1

Tipos de sesiones Beans

- Existen tres tipos de sesiones:

1. **Stateful - Con estado:** El estado del bean se mantiene a través de varias llamadas a métodos.

   El "estado" se refiere a los valores de sus variables de instancia. Debido a que el cliente interactúa con el bean, este estado se llama a menudo el estado *de conversación.*

Tipos de sesiones Beans

2. **Stateless:** Los beans sin estado se utilizan para las operaciones que se pueden producir en una sola llamada al método. Cuando el método termina el procesamiento, no se conserva el estado del bean específico del cliente.

Tipos de sesiones Beans

3. **Singleton:** Un bean de sesión singleton se instancia una vez por aplicación, y existe para el ciclo de vida de la aplicación.

   Esta sesión está diseñada para ser compartida y accedida concurrentemente por los clientes

Sesion Bean JPA

- Los beans de sesión proporcionan servicios (transacciones, seguridad, control de acceso) a una aplicación cliente.
- Las entidades representan los datos de negocio.



Patrón Facade

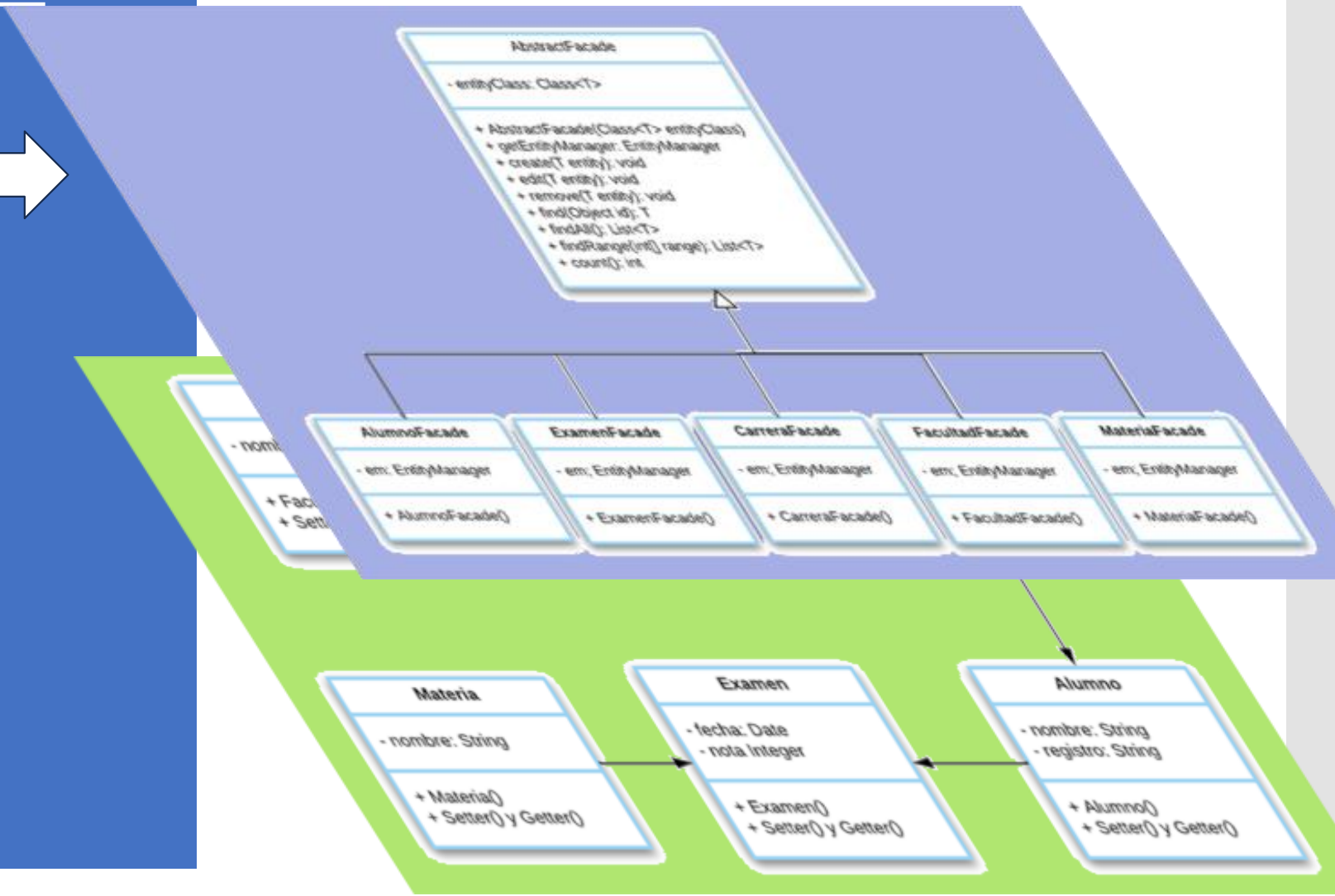
Un bean de sesión proporciona una interfaz conveniente para la manipulación de las entidades (crear, recuperar, actualizar y eliminar) – CRUD -

Sesión Facade

Características:

- Es un patrón de diseño que, intenta resolver los problemas comunes que surgen en un entorno de aplicaciones de varios niveles, tales como:
  - Acoplamiento lo que conduce a la dependencia entre los clientes y los objetos de negocio.
  - Demasiadas llamadas a métodos entre el cliente y el servidor, lo que lleva a problemas de rendimiento de la red
  - La falta de una estrategia uniforme de acceso de cliente, exponiendo los objetos de negocio a un mal uso

Aplicación


AbstractFacade
- entityClass: Class<T>
+ AbstractFacade(Class<T> entityClass)
+ getEntityManager(): EntityManager
+ create(T entity): void
+ edit(T entity): void
+ remove(T entity): void
+ find(Object id): T
+ findAll(): List<T>
+ findRange(int[] range): List<T>
+ count(): int
AlumnoFacade
- em: EntityManager
+ AlumnoFacade()
ExamenFacade
- em: EntityManager
+ ExamenFacade()
CarreraFacade
- em: EntityManager
+ CarreraFacade()
FacultadFacade
- em: EntityManager
+ FacultadFacade()
MateriaFacade
- em: EntityManager
+ MateriaFacade()
Materia
- nombre: String
+ Materia()
+ Setter() y Getter()
Examen
- fecha: Date
- nota Integer
+ Examen()
+ Setter() y Getter()
Alumno
- nombre: String
- registro: String
+ Alumno()
+ Setter() y Getter()

Conclusión

Las tecnologías vistas funcionan de manera integrada para configurar rápidamente el modelo de cada aplicación, conservando las pautas propuestas por el patrón MVC.

App Java Enterprise

Incluye:

- Proyecto de una Aplicación Web
- Diseño de la interfaz de usuario
- Modelo de Datos
- Conexión a MySQL
- Generación de Entidades
- **Generación de Sesiones Beans**
- **Diseño con patrones, Model-View-Controller (MVC) y Session Facade**

Agregar sesiones

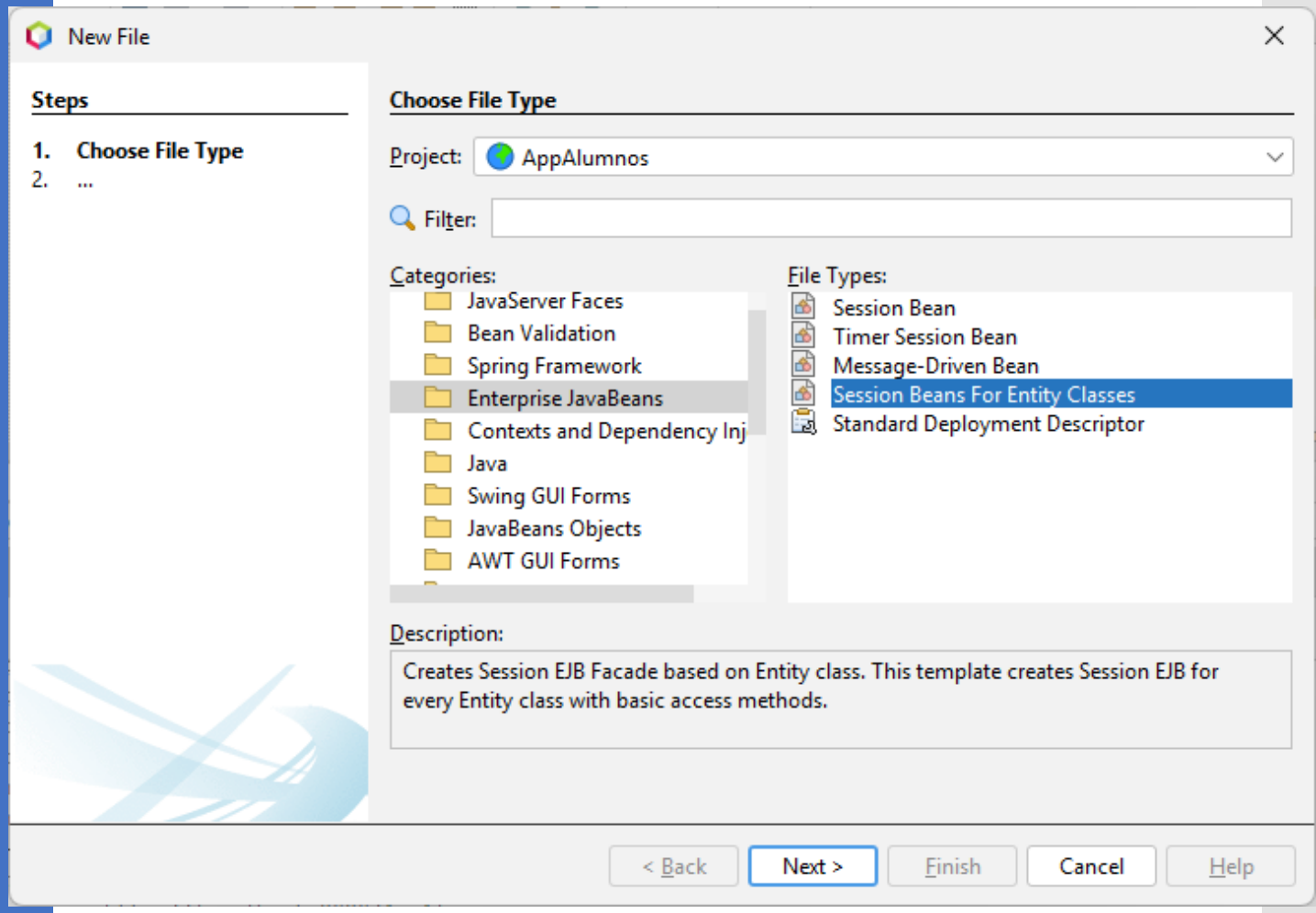
El asistente genera una "fachada" sesión EJB (patrón Facade) para cada una de las clases de entidad que se han creado. Cada sesión bean contendrá métodos de acceso básicos para su respectiva clase de entidad.

Sesión Facade

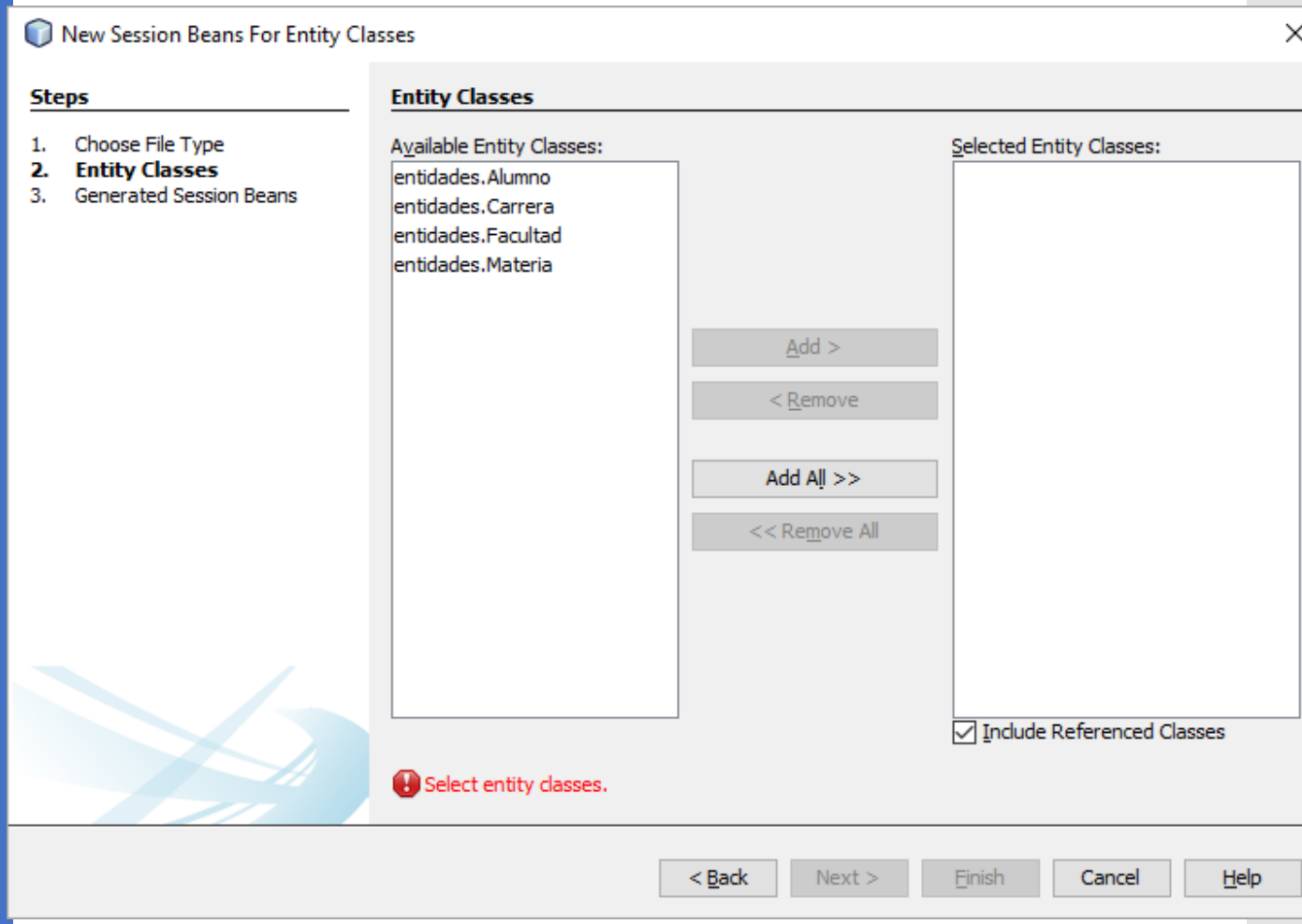
Características:

- Abstrae las interacciones de objetos de negocio. Proporciona una capa de servicio que expone sólo la funcionalidad requerida.

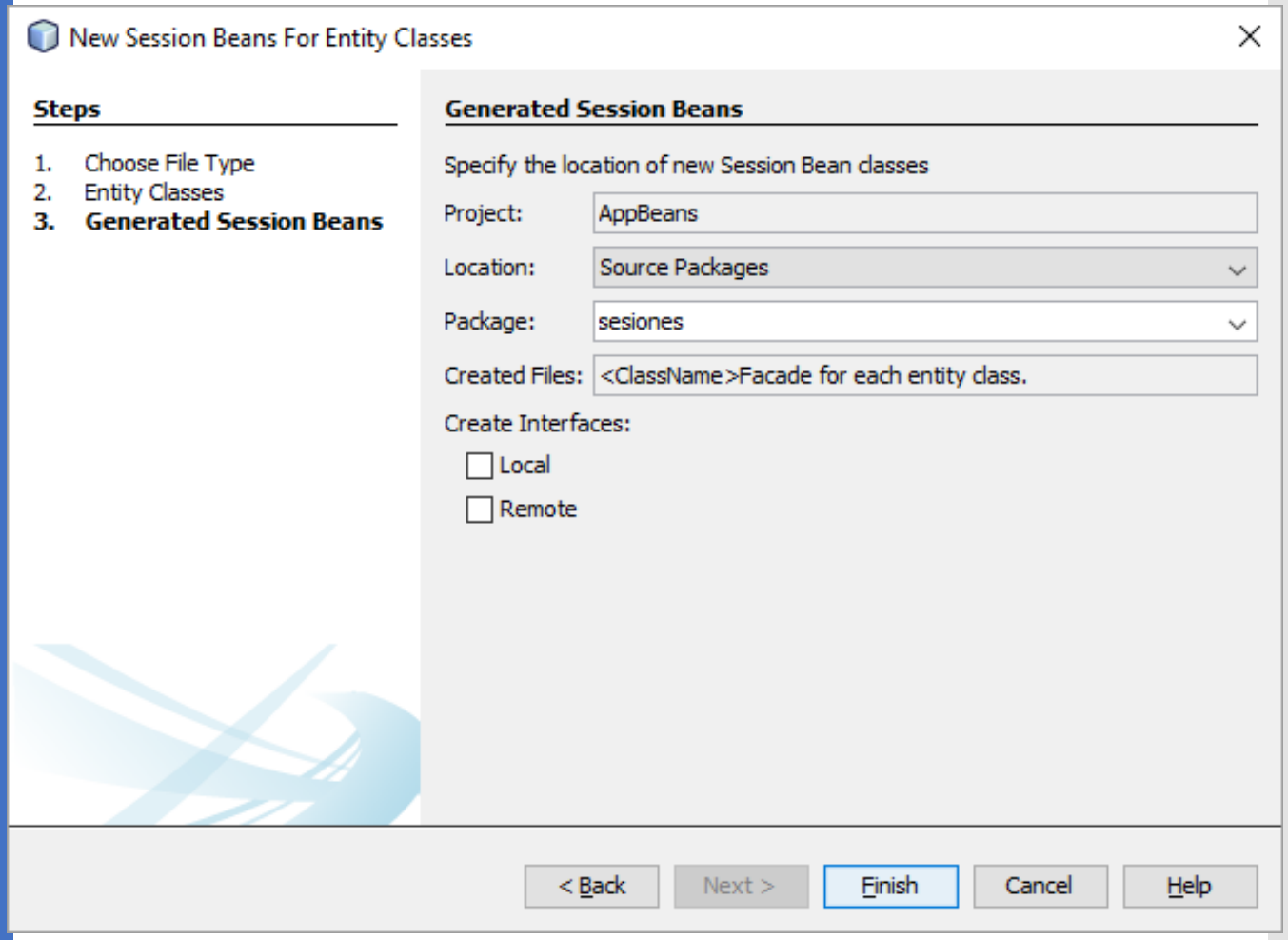
Se crea un nuevo componente

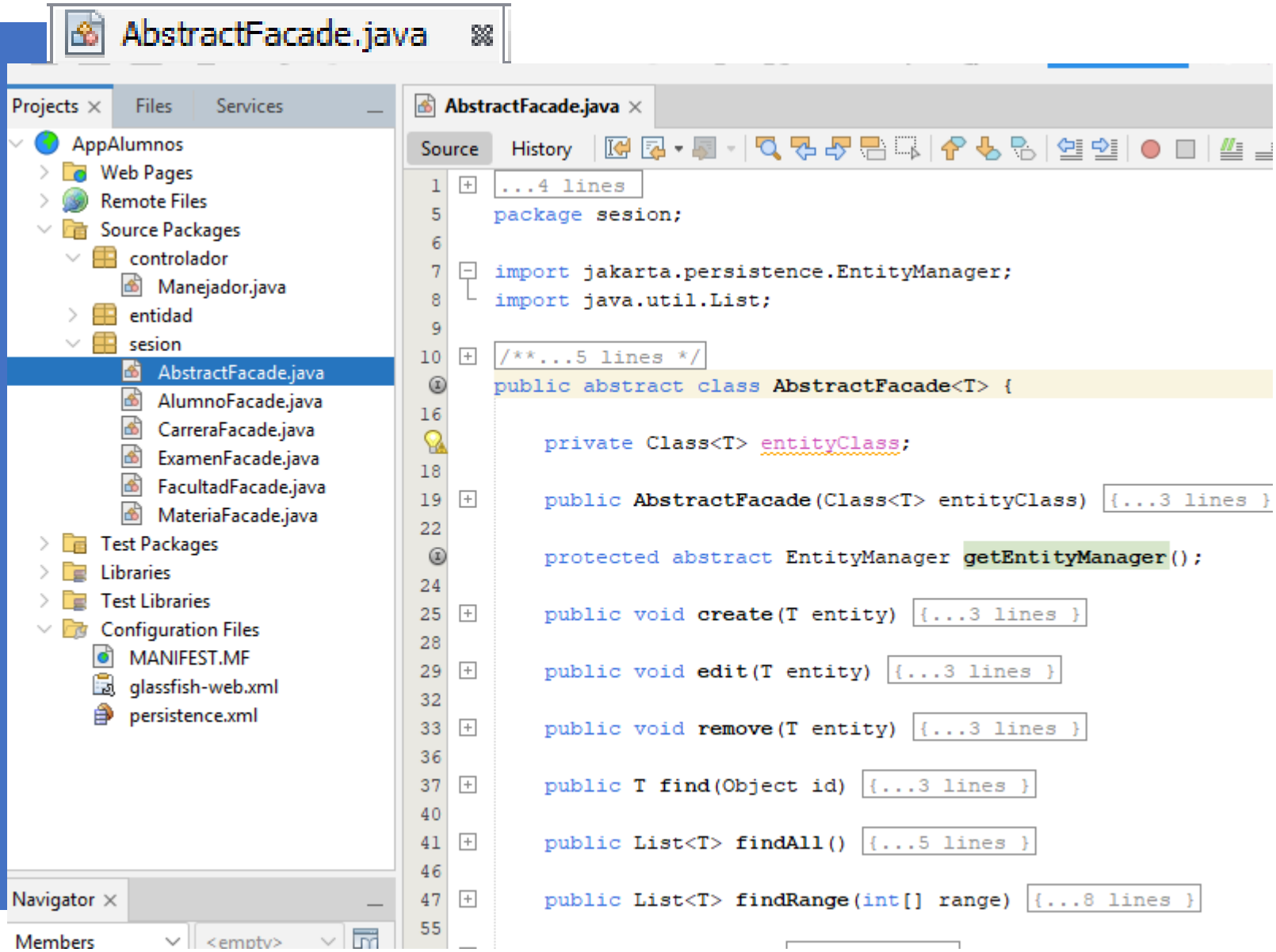


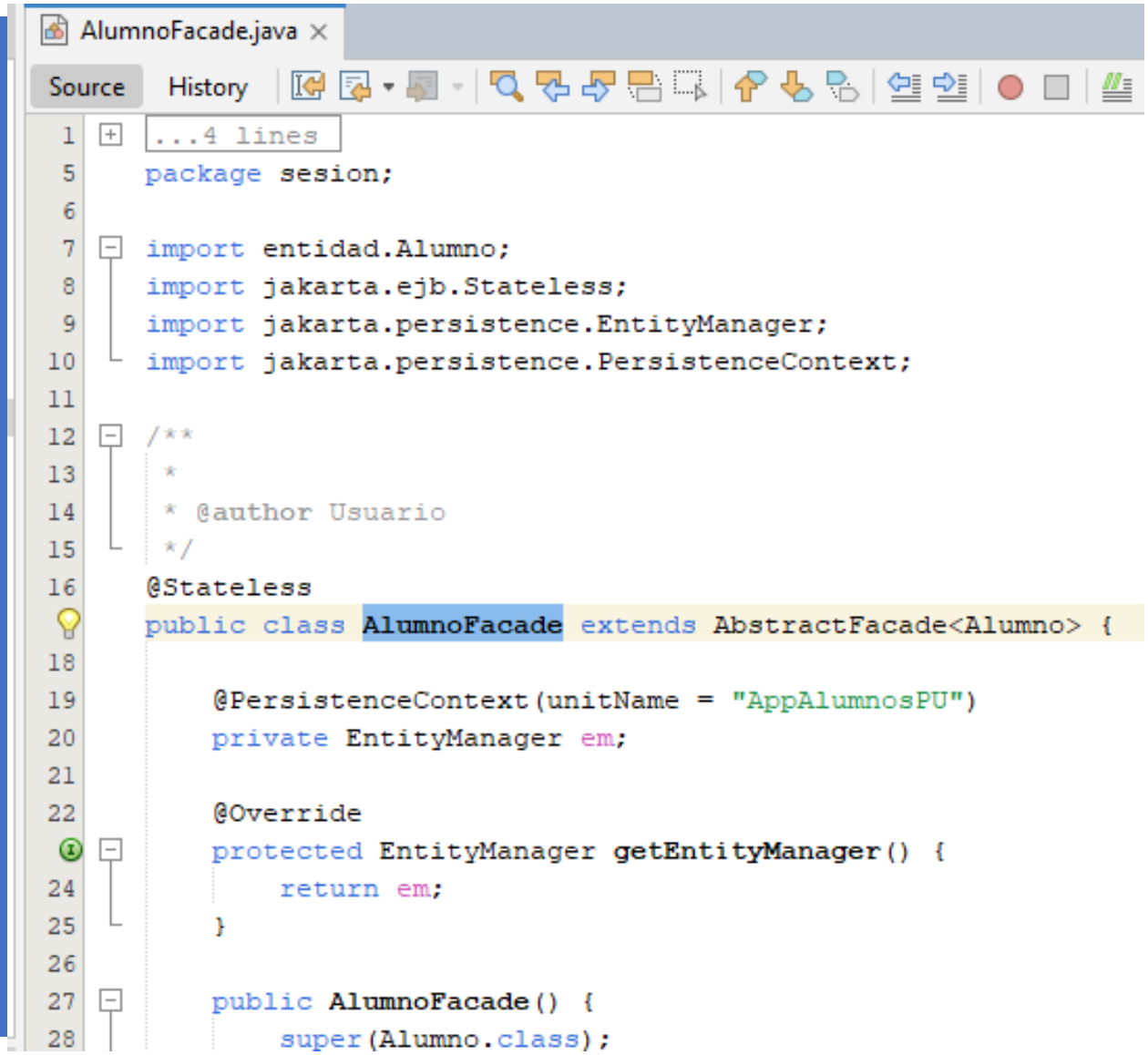
- Se eligen todas las entidades



- Se ingresa el paquete




AbstractFacade.java
Projects
Files
Services
AppAlumnos
Web Pages
Remote Files
Source Packages
controlador
Manejador.java
entidad
sesion
AbstractFacade.java
AlumnoFacade.java
CarreraFacade.java
ExamenFacade.java
FacultadFacade.java
MateriaFacade.java
Test Packages
Libraries
Test Libraries
Configuration Files
MANIFEST.MF
glassfish-web.xml
persistence.xml
Navigator
Members
<empty>
AbstractFacade.java
Source
History
...4 lines
package sesion;
import jakarta.persistence.EntityManager;
import java.util.List;
/**...5 lines */
public abstract class AbstractFacade<T> {
private Class<T> entityClass;
public AbstractFacade(Class<T> entityClass) {...3 lines }
protected abstract EntityManager getEntityManager();
public void create(T entity) {...3 lines }
public void edit(T entity) {...3 lines }
public void remove(T entity) {...3 lines }
public T find(Object id) {...3 lines }
public List<T> findAll() {...5 lines }
public List<T> findRange(int[] range) {...8 lines }


AlumnoFacade.java
Source
History
...4 lines
package sesion;
import entidad.Alumno;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
/**
*
* @author Usuario
*/
@Stateless
public class AlumnoFacade extends AbstractFacade<Alumno> {
@PersistenceContext(unitName = "AppAlumnosPU")
private EntityManager em;
@Override
protected EntityManager getEntityManager() {
return em;
}
public AlumnoFacade() {
super(Alumno.class);

- @PersistenceContext se utiliza para inyectar un EntityManager gestionada por contenedor en la clase.
- En este caso se basa en el contenedor EJB GlassFish para abrir y cerrar EntityManagers como y cuando sea necesario.
- El elemento unitName especifica la unidad de persistencia AppBeansPU, que ha sido definido en el archivo de persistence.xml de la aplicación.

```java
public class Manejador extends HttpServlet {

    @EJB
    private AlumnoFacade alumnoF;
```

```java
        case "/Listar":
    /*      em = emf.createEntityManager();
            Query q = em.createNamedQuery("Alumno.findAll");
            List todos = q.getResultList();
*/
            request.setAttribute("lista",alumnoF.findAll());
        //   getServletContext().setAttribute("lista",emf.findAll());
            url = "/WEB-INF/vista/" + pathUsuario + ".jsp";
            break;
```

ListarAlumnos.jsp

```
13  <html>
14      <head>
15          <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
16          <title>JSP Page</title>
17      </head>
18      <body>
19          <h1>Listado de alumnos</h1><hr>
            <table id="lista" border="3">
<tr>
            <th bgcolor="red">Registro</th>
            <th bgcolor="red">Nombre</th>
24          <th bgcolor="red">Carrera</th>
25  </tr>
26  <c:forEach var="a" begin="0" items="${lista}">
27  <tr>
28          <td>${a.getRegistro()}  </td>
29          <td>${a.getNombre()}  </td>
30          <td>${(a.getCarreraIdcarrera()).getNombre()}  </td>
31  </tr>
32
33  </c:forEach>
34
35  </table>
36
37      </body>
38  </html>
39
```

Segunda parada

1. Generar los Beans en tu aplicación.
2. Dejar a punto y desplegar.
3. Realizar los cambios necesarios para que aparezca en el listado de alumnos el nombre facultad.



Ejercicio

- Crear la aplicación y los componentes necesarios para que funcione el **Sistema Alumnos.**

Recursos

- Que es Apache Derby?
- Anotaciones
- Tutorial Java EE5
- Tutorial Java EE6
- API documents
- Especificaciones Derby